六、C++相关（伍征、邓天宇老师）

6.1 说一下指针和引用的区别

引用的本质是指针常量，引用可以做到的事都可以使用指针做，但引用更安全，更简洁。而指针更灵活，更通用。

	指针	引用
初始化	可以为 NULL，或者不初始化用随机值	初始化必须引用一个已有确切的对象
sizeof	指针变量的大小固定为 4Byte / 8Byte	被引用的对象的大小
类型	弱类型，允许跨类型转换，存在多级指针	强类型，不允许跨类型引用，只存在引用传递
优点	程序灵活，操作连续内存快	安全，避免了传参从而提高了效率
缺点	需要防止指针越界访问数据	引用关系不能切换，灵活性较差

扩展：什么时候使用指针？什么时候使用引用？

引用往往用在函数传参传递参数本体，可以提升传参性能，也可以成为出参。引用也可以作为函数返回值，不过返回的内容在函数结束后不能被释放。而指针可以用来遍历、管理内存。

6.2 new/delete 和 malloc()/free()的区别

	new/delete	malloc()/free()
操作本质	C++下的运算符，可以通过重载运算符的方式修改其功能；	C 库提供的内存管理函数；
错误处理	如果 new/delete 出错，则会抛出异常 throw，需要用户捕获异常 catch；	通过返回值的形式告知用户成功与否，往往需要释放先前的资源后结束程序；
空间特性	在自由存储区开辟空间；开辟的空间是有类型的。以 Type *的形式告知用户；	在堆上开辟空间； 开辟的空间是以 void *形式告知用户的；
操作细节	new 开辟空间而且直接调用构造函数，使空间中有数据；delete 释放的空间不可再使用，一旦使用直接报错。	malloc()开辟空间，但空间中是没有数据的； free()释放的空间是不可继续使用，但意外使用的话不会报任何错误，只是数据不确定。
其它	new 还可以用来定位内存，给一块已经申请过的	

华清远见武汉中心嵌入式学科面试宝典

	内存赋予新的解释方式	

6.3 高内聚低耦合的概念是什么

高内聚低耦合是一种编程思想（或者说规范），它就要求开发人员在面向对象编程的过程中尽可能做到：类的内部功能关联可以很复杂，但对外接口尽量单一；且类的封装中应该尽可能减少对外部对象的依赖，一个类只实现某类或某个方法。

高内聚低耦合的优势：

1）增强了模块代码的复用性，方便移植单个模块相关文件到其他工程，而不需要做过多的修改工作；

2）高内聚低耦合使得项目中模块与模块之间的关系尽可能的简单直接，这给项目的调试、分工、维护带来的好处是巨大的；

降低耦合度的方法：

1）少使用类的继承，多用接口隐藏实现的细节；

2）界面类与业务逻辑类应当做到分离，尽量避免复杂的依赖关系；

3）遵循一个定义只在一个地方出现，并且少使用全局变量；

5)如果模块间必须存在耦合，应当考虑用设计模式加以优化，实在不能优化也要尽可能做到单向依赖。

6）尽量不用"硬编码"的方式写程序，同时也尽量避免直接用 SQL 语句操作数据库

增强内聚度方法：

1）模块只对外暴露最小限度的接口，类属性和方法的访问权限能用 private 就不要用 public；

2）模块的功能化分尽可能的单一，过度依赖的两个类应该合并为一个类；

3）模块内部的修改，不得影响其他模块，如果有影响就要想办法解决；

6.4 C 和 C++中 struct 和 class 有什么区别

在 C 语言中，struct 默认为公有权限，且没有权限修饰符，只能通过特殊技巧模拟私有成员，此外不能定义成员函数，但可以使用函数指针。

C++中，struct 默认为公有权限，其它功能与 class 一致，可以使用权限修饰符，可以定义成员函数。

C++中，class 默认为私有权限。

华清远见武汉中心嵌入式学科面试宝典

6.5 内联函数与宏函数的区别

宏函数是在**预处理阶段**进行的代码替换。

内联函数在**编译器阶段**是直接复制"镶嵌"到主函数中去的，就是将内联函数的代码直接放在内联函数的位置上，所以没有指令跳转，指令按顺序执行。**内联函数运行速度比常规函数稍快，但代价是需要占用更多的内存**。

内联函数是真正的函数。需要进行类型检查，而宏函数不是真正的函数，因此不会进行类型检查。

6.6 C++中 static 关键字的作用以及何时使用 static

注意 **static 在 C 语言中的点也需要回答**，在 C 语言部分查看对应知识点。

在 C++中 static 可用于**修饰成员变量**，被修饰的成员变量需要在类内声明，类外初始化。并包含以下特点：

- 被修饰的成员变量属于类不属于具体的对象。
- 静态成员变量的内存在程序开始时分配，程序运行结束时释放内存。
- 静态成员变量对于所有该类的成员是共享的，并在对象创建之前就已经产生了。
- 静态成员必须进行初始化，否则会在 linker（链接）步骤时报错
- 使用静态成员可以不需要定义出对象，可以通过类名＋域操作符访问

static **修饰成员函数**有以下特点：

- 与修饰静态成员变量一样，可以通过类名 + 域操作符访问
- 与类关联，不与对象关联，不能使用 this 指针
- 不能访问非静态成员（成员函数与成员变量），只能访问静态成员（成员函数与成员变量）

何时使用 static 关键字？（大多面试会深入问具体使用位置）

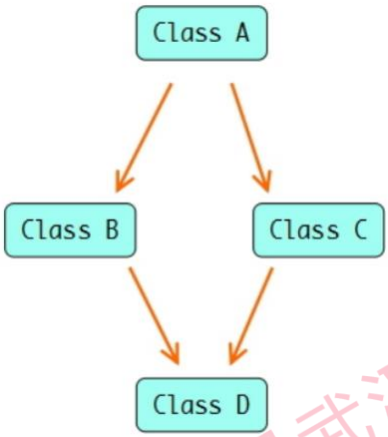
1）**一切不需要实例化（创建对象）就可以确定行为方式的函数（一般被称作工具函数）都应该设计为静态的**。例如：圆类中计算面积和周长的函数；简单工厂中创建对象的函数。

2）**通过其它类型对象转换成自己类型对象一般使用静态成员函数**。例如：Qt 中图片类型转换函数；不同类型字符串格式转换函数。

3）单例的设计。

华清远见武汉中心嵌入式学科面试宝典

6.7 菱形继承引发的问题，以及解决方案



引发问题

如图类 B 与类 C 继承类 A，而类 D 多继承与类 B 与类 C，此时会引发数据冗余与二义性的问题。

数据冗余：在创建 D 类的对象时，类 A 的构造函数将会调用两次，相当于创建两个类 A 对象

二义性：A 类中成员变量，可以通过 B 和 C 去访问，此时会存在两个同种含义的变量

解决方案

1． 尽量选择避免这种设计，本身这种设计较为繁琐且对性能也有所影响。

2． 使用虚继承，在类 B 与类 C 继承类 A 时使用虚继承即可解决这种问题

注意：在**虚继承的设计中，无需在中间类对基类进行初始化**，如果类 B 或类 C 在初始化列表中初始化了类 A，**此时类 B 与类 C 的初始化列表中针对类 A 的初始化会失效，会调用类 A 的无参构造函数**，若想人为选择类 A 的构造函数，需要在类 D 的初始化列表中去选择类 A 的构造函数初始化

6.8 什么是多态，如何实现多态，请举例说明

多态指**同种行为，不同的对象有不同的表现**。

在 C++中多态分为**静态多态 (编译多态)** 与**动态多态 (运行多态)** 两种，实现静态多态可以通过函数重载或泛型编程，实现动态多态需要用到继承与虚函数。

函数重载指在同一作用域，相同的函数名，不同的参数列表构成重载函数，在调用函数时，编译器会根据传参类型选择要调用的函数。如果存在隐式类型转换，那么编译器与用户所期待的函数调用会存在差异，因此需要用户明确知晓自己传递的参数类型。

泛型编程需要创建对应的模板，针对特殊类型也可以对模板进行显式实例化。

动态多态的实现的基础是**子类重新实现父类的虚函数**，调用时为**父类指针或引用指向子类对象，通过该指针或引用调用被子类所重新实现的虚函数**。请注意，如果该虚函数没有被子类重新实现、或者该虚函数为子类新增函数，并没有出现在父类中，那么仍然不会触发动态多态。

华清远见武汉中心嵌入式学科面试宝典

6.9 一般推荐把父类的析构函数设置成虚函数，为什么？

根据语法规则，将父类的析构函数设置成虚函数，那么子类的析构函数会自动变为虚函数。

当**父类指针指向子类对象**的时候，如果析构函数不是虚函数，则不会发生动态多态（运行时多态），而导致**只会调用父类的析构函数，只会释放掉对象中父类的资源，此时子类对象资源没有释放，从而导致内存泄漏**。

扩展：动态多态的实现原理由**虚函数表**实现，**一旦类中引入了虚函数，在程序编译期间会创建虚函数表，表中每一项数据都是虚函数的入口地址**，为了将对象与虚函数表关联起来，编译器会在**对象中会增加一个指针成员用于存储虚函数表的位置，基类的指针指向派生类对象时就是通过虚函数表的指针来找到实际应该调用的函数**。

基类与派生类都维护自己的虚函数表，虚函数表位于只读数据段(.rodata)，如果派生类重写基类的虚函数，则派生类的虚函数表存储的是派生类重写过后的虚函数地址。如果没有重写，那么派生类的虚函数表中存储的是基类的虚函数地址。

6.10 简述覆盖（重写/覆写）、重载、隐藏的概念

覆盖/重写（override）

派生类重新实现基类的虚函数，要求必须处于具有继承关系的类中（直接继承与间接继承）、函数名、参数、返回值都必须相同，基类中该函数必须要有 virtual 关键字修饰，重写函数的权限访问限定符可以不同，通过父类指针或引用指向子类对象，再去调用该函数。主要目的是用于实现动态多态。原理是使用虚函数表。

覆盖/重写存在例外，第一个例外为析构函数，析构函数只需要加上 virtual 即可成为虚函数，第二个例外为协变，要求返回值类型可以不相同，但是必须是基类/派生类的指针或者引用。

重载（overload）

同一作用域中的函数名相同，参数不同的多个函数间构成重载。主要目的是提高的易用性，减少函数名数量，同时提高程序的可读性，原理是编译器会将重载函数设置成不同的函数名，根据参数类型与个数进行匹配

扩展问题：在 C++ 程序中调用被 C 编译器编译后的函数，为什么要加 extern "C"？

C++语言支持函数重载，C 语言不支持函数重载。函数被 C++编译后在库中的名字与 C 语言的不同。

华清远见武汉中心嵌入式学科面试宝典

假设某个函数的原型为：void foo(int x, int y);

该函数被 C 编译器编译后在库中的名字为 _foo，而 C++编译器则会产生像_foo_int_int 之类的名字。

C++提供了 C 连接交换指定符号 extern"C"来解决名字匹配问题。

隐藏（hiding）

父类与子类有同名函数，调用的时候总是调用子类的函数，此时父类成员函数被隐藏。主要目的是一般子类继承过来的函数不适合子类，或者需要扩展 ，则需要重写父类的函数。两个函数的返回值和参数可以相同也可以不同。

6.11 STL 中 vector 与 list 的区别

vector 与 list 都属于 STL 中的**序列式容器（顺序容器），元素以严格的线性形式组织起来，每个元素都有固定位置。**

vector 本质是动态数组，随机存取任何元素都能在常数事件完成，在尾端增删元素性能高。增加数据时大致按以下流程：

建立空间->填充数据->重建更大空间->复制原空间数据->删除原空间->添加新数据

list 本质是双向循环链表，在任何位置增删元素都能在常数时间完成，但随机访问偏慢。

主要区别在于 vector 支持下标操作，而 list 不支持，**vector 多用于存储已知长度(模糊范围也可)且经常随机访问的数据，list 主要用于存储长度未知且经常增删数据而少量随机访问。**

6.12 简述什么是迭代器？有什么作用？迭代器失效是什么意思？

迭代器是一种检查容器内元素并遍历元素的数据类型，迭代器的核心作用为**使算法独立于容器类型。**标准库为每一种标准容器定义了一种迭代器类型，而极少数容器支持下标操作访问容器元素。

由于一些对容器的操作如删除元素或移动元素会修改容器的内在状态，会使原本指向被移动元素的迭代器失效，也可能使其他迭代器失效。**使用无效的迭代器是没有意义的，可能会导致和使用空指针相同的问题，**所以使用迭代器时，需要特别留意哪些操作会使迭代器失效，**使用无效迭代器会导致严重的运行错误。**

6.13 什么是智能指针？简述智能指针实现原理

所谓智能指针就是**智能/自动化的管理指针所指向的动态资源的释放。**它是一个类模板，有类似指针的功能，对*和->运算符进行了重载。

华清远见武汉中心嵌入式学科面试宝典

实现原理：

智能指针用一个类描述，这个类中有一个指针成员(一个引用计数成员)，**构造函数中初始化指针成员指向对象**(初始化引用计数成员的值为 1)，**析构函数中删除指针成员指向的对象**(将引用计数的值自减，如果减到 0 的时候，删除指针成员指向的对象)

扩展：

智能指针并非线程安全，如果想让他线程安全可以选择继承该类，重写对应功能，增加互斥锁使之线程安全。另外 **shared_ptr 在相互引用的情况下会出现故障**，从而需要引入 weak_ptr 协助 shared_ptr 工作。

6.14 什么是单例？如何实现单例？

该类型只有一个对象，不能再额外创建一个新的对象

实现步骤：

1. 将构造函数访问权限设置成 private 或者 protected ，不允许类的外部创建对象
2. 用一个静态成员指针指向创建的对象
3. 提供一个静态成员函数，获取这个对象的首地址

饿汉式与懒汉式:饿汉式单例在第 2 步时直接 new 出该对象，懒汉式单例在第 2 步时令指针等于 NULL，只有在第一次用到类实例的时候才会去实例化

懒汉式线程不安全问题：两个线程同时调用获取单例的静态函数时可能造成内存泄漏，故需要在该函数中增加互斥锁，但是在已创建此对象时再加锁会影响程序性能，所以在加锁前需要再次判断是否已经创建出对象。

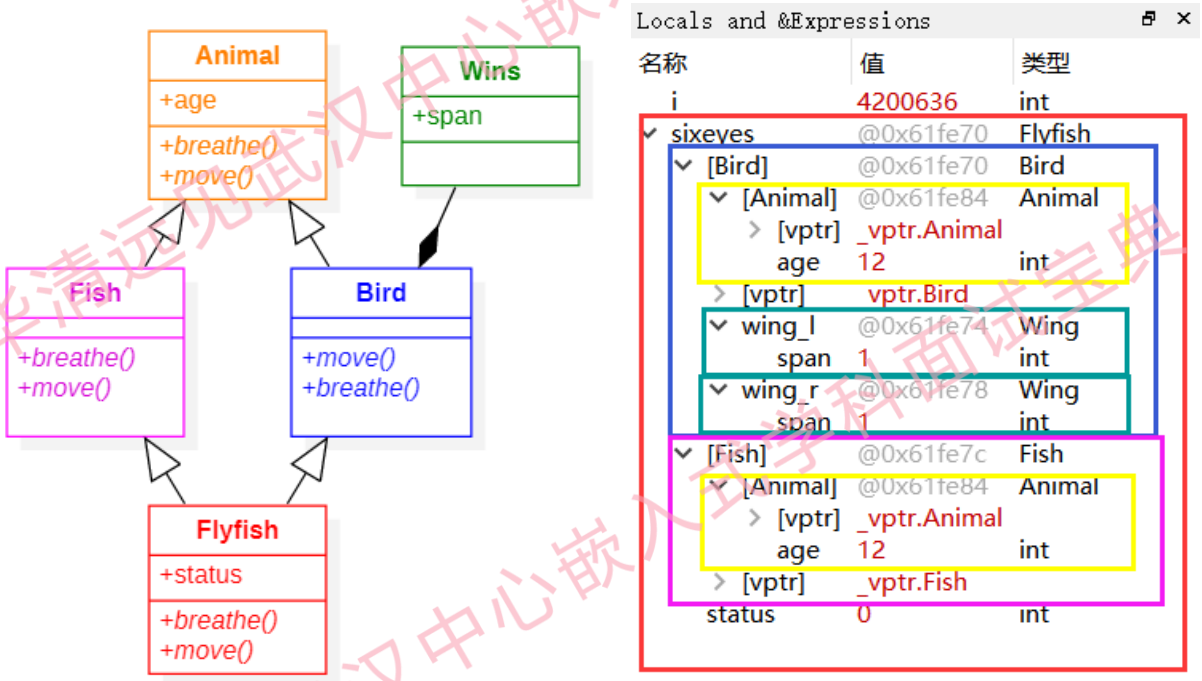
如何自动释放单例：可以使用智能指针管理对象的内存，将智能指针定义在静态区即可；也可以在单例中创建一个内部类，内部类的析构函数中释放单例，内部类对象创建在静态区，那么在程序结束时则自动释放掉单例。

注意：其它常考的设计模式还有工厂、策略模式、代理模式、适配器模式、观察者模式、组合模式，建议学习其原理，并能绘制 UML 类图

华清远见武汉中心嵌入式学科面试宝典

6.15 C++对象的内存布局是什么，分为哪些情况

首先，对象的内存分布布局是指对象空间中的内存组成，一般的类由成员变量组成（成员函数并不占用对象的内存空间）。但 C++中的 class 不是 C 语言的 struct，它不仅仅有嵌套（C++下称作成员对象）还有继承、虚继承、多重继承的情况。简单来说越是偏向派生类的实体对象，其内存布局越复杂，包含的基类数据也就越多。请看下面这个复杂的例子：



飞鱼类 Flyfish 是最大的类，它以继承的方式继承了鱼类 Fish 和鸟类 Bird（先不管是否符合事实），然后鱼类 Fish 和鸟类 Bird 以虚继承的方式继承了动物类 Animal；这其中鸟类 Bird 以组合的方式包含了 2 个翅膀类 Wins。他们在内存中的关系图如上右图所示（这便是对象的内存分布）。注意以下几点：

1）只要存在继承链上出现了虚函数，那么内存布局中就有 vptr 这个指针，它指向了虚函数表，它是调用子类虚函数方法必要的指针；

2）右图中 2 处出现了动物类 Animal 的分布，但其地址是一样的，说明只有一个动物类的数据存在于内存中；（虚继承避免菱形继承在此时就得到了体现）

3）内存分布中的成员类或者父类的空间块排列顺序和构造顺序其实是一致的；

4）如果 Flyfish 类还包含了一个成员类 A，那么成员类 A 的空间一定在成员变量 status 前面，这就是由构造顺序决定；

5）静态成员变量处于静态区（类似于堆空间），不属于任何一个对象实体的内存空间！

华清远见武汉中心嵌入式学科面试宝典

6.16 C++11 有什么新特性，简述你所知道的新特性

1. nullptr 指针：用于代替 NULL，解决部分编译器将 NULL 直接解释成 0，导致函数重载引发的调错函数问题（int 类型与指针类型的问题解决），但是不能解决指针类型的函数重载问题（int * 与 char *之间仍然会出现问题）
2. 列表初始化：统一初始化的语法格式，可以使用 { } 对所有类型进行初始化
3. 别名声明 using：可用于代替 typedef 给类型取别名
4. 类型推断：auto 关键字与 decltype 类型指示符。用于定义变量时不确定类型的情况，让编译器推断，其中 auto 常用于推导变量类型，decltype 常用于推导表达式类型。
5. **右值：右值引用用于解决非常量左值引用无法接收右值的情况，另外引入了万能引用与完美转发**
6. **完美转发主要是由三部分构成，第一部分为万能引用，需要结合泛型编程一起使用，用来接收所有类型。第二部分为引用折叠规则，详细规则可以上网搜素，简单来说我们可以认为如果出现了左值，那么整个变量类型将会被推导成左值引用，否则将会被推导成右值引用。第三部分为 std::forward 函数，主要用来还原对象在进入函数前本身的值类型（注意与变量类型区别）**
7. 移动构造函数：通过右值实现，目的是解决深拷贝针对于临时对象浪费过多资源的问题。
8. 委托构造函数：通过初始化列表委托其它构造函数进行初始化。
9. **范围 for 循环：主要用于各种容器的遍历**
10. **拉姆达表达式（匿名函数）：可以直接在需要调用函数的位置定义短小精悍的函数，不需要预定义好函数，使代码更加紧凑，结构层次更加明显、可读性更好**

注：特性不止这些，只列举了相对常见部分，加粗的部分尤为重要

华清远见武汉中心嵌入式学科面试宝典